AspiroBrain

Data Pipeline — Implementation Plan

From scraped CSVs to a trustworthy, hallucination-resistant data foundation

Prepared by Ahsanul Hoque | AspiroWork Data Internship | 10 July 2026

Document Purpose

This document translates the *"Research Note: Data Challenges in Building AspiroBrain"* into a phased, engineering-ready implementation plan. It is grounded in the scraping pipeline already running today (Netherlands and Malta master's program data, 200 records, `mastersportal.com`), and proposes the concrete architecture, technology choices, and rollout sequence needed to turn that data into a foundation the AspiroBrain LLM assistant can trust.

Contents

1 Executive Summary

AspiroBrain's success depends less on the LLM than on the data feeding it. The research note identifies seven concrete failure modes — wrong currency, inconsistent formats, silent gaps, stale data, duplicate programs, untrustworthy sources, and unmanageable scale — each of which can translate directly into a student missing a deadline, misjudging a budget, or losing trust in the platform.

This plan proposes a four-phase build, each with an explicit trigger for moving to the next phase, so effort is never spent ahead of actual need:

- **Phase 0 — Fix now.** Add three missing columns to the existing CSV pipeline. Half a day of work, removes the most expensive-to-backfill-later gaps.
- **Phase 1 — Automate.** Turn manual scrape sessions into a scheduled, self-validating pipeline with change detection, so re-verification cost stops scaling linearly with catalog size.
- **Phase 2 — Structure.** Replace the flat CSV with a relational data model that gives every program a stable identity and every fact a source, a trust tier, and a freshness timestamp.
- **Phase 3 — Ground the LLM.** Wire AspiroBrain's retrieval and response generation so it can only state facts it actually retrieved, and is architecturally incapable of inventing a tuition figure or a deadline.

Phases 0–1 require no new infrastructure and can start immediately inside the current workflow. Phases 2–3 require an infrastructure and timeline decision from the team — flagged explicitly in §8.

2 Problem Statement

The research note identifies seven data challenges. Table 1 maps each to its root cause and the concrete student-facing consequence if left unaddressed — this is the "why" behind every architectural decision in §3.

Challenge	Root Cause	Student-Facing Risk
1. Multi-currency	Tuition published in EUR/GBP/USD/CAD/…; students think in BDT	Stored conversions go stale as FX rates move; budget looks wrong
2. Heterogeneous sources	Same fact phrased differently across sites ("Fall Intake" vs "Autumn Semester")	Duplicate or missed results; poor search quality
3. Missing data	Universities publish fields gradually	LLM invents a plausible but false answer (hallucination)
4. Data freshness	Tuition/deadlines/requirements change yearly	Recommends an outdated fee or a passed deadline
5. Program identity	Universities rename/rebrand programs	Duplicate listings; broken continuity across re-scrapes
6. Source governance	Conflicting values across sources of unequal reliability	System has no basis to decide which number to trust
7. Scalability	Manual verification does not scale to hundreds of universities	Data quality degrades exactly as the catalog grows

Table 1: The seven data challenges, their cause, and their downstream risk.

Current State — 10 July 2026

Two country CSVs (Netherlands, Malta; 200 programs total), single source (`mastersportal.com` — Tier 5 of 6 on the trust hierarchy in §6). Known schema gaps: no `currency` column, no `last_verified_date`, no `program_id`; repeatable fields flattened into semicolon-separated text. Two real bugs already caught and fixed by hand during scraping: *tuition front-loading* (headline "€X/year" is sometimes an average across an uneven multi-year fee schedule) and *scholarship-widget noise* (a sitewide generic scholarship list bleeding into program-specific tags).

3 Solution Architecture

Figure 1 shows the full pipeline as four layers, each corresponding to one implementation phase. Data flows top to bottom: raw pages are scraped and frozen, then cleaned and quality-checked, then loaded into a structured store with identity and trust metadata, then served to the LLM through an API that enforces freshness and handles currency conversion live.

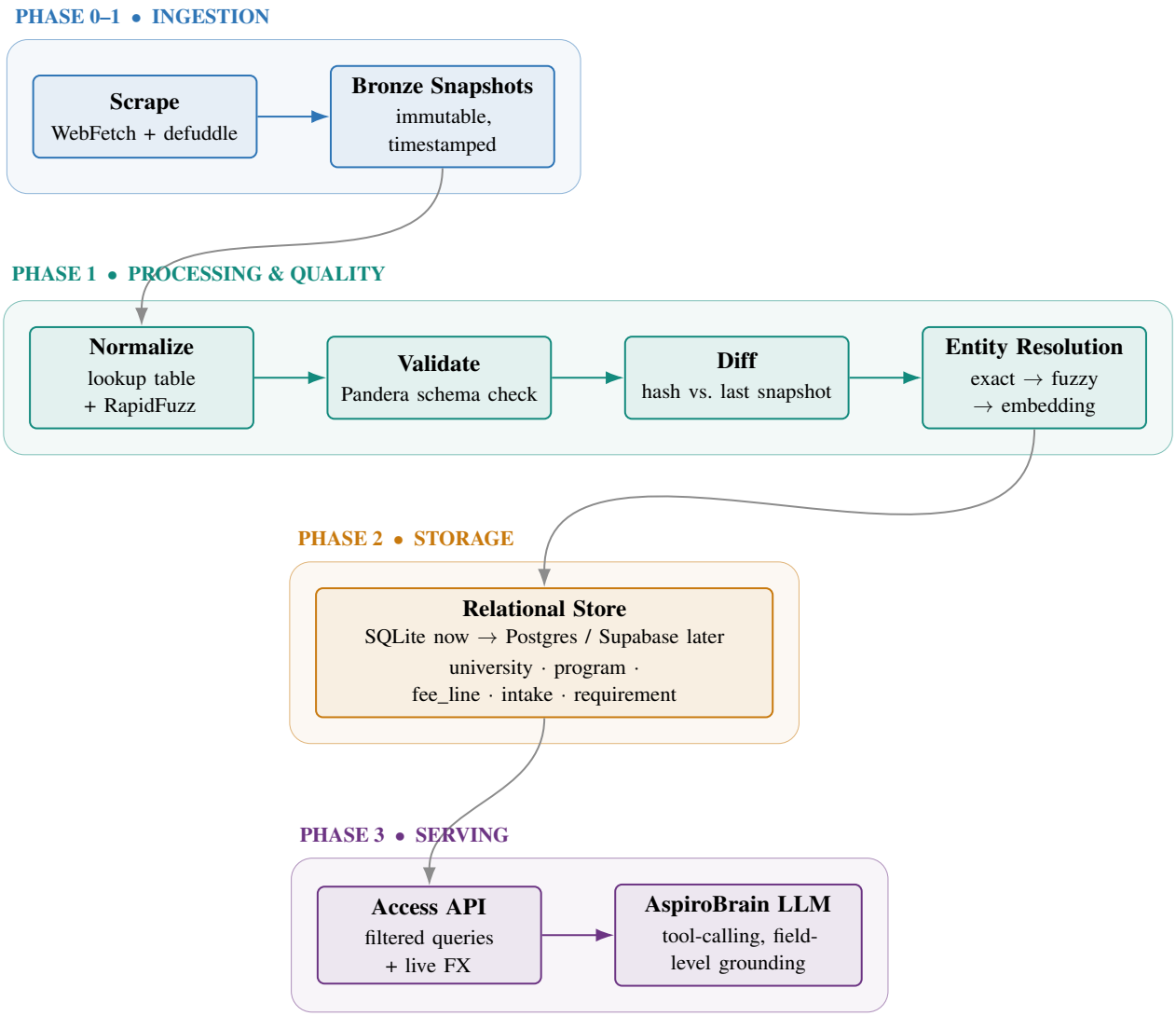

Figure 1: AspiroBrain data pipeline: four layers, each mapped to an implementation phase. Every arrow closes at least one of the seven challenges in Table 1 — normalization resolves heterogeneous formats, the diff step resolves freshness, entity resolution resolves program identity, and the storage layer's trust-tier and verification fields feed directly into how the LLM is allowed to answer.

3.1 Why hybrid retrieval, not vector-search-everywhere

Structured questions (country, budget, degree level, deadline) are answered with a direct filtered query against the relational store — no embeddings involved, no risk of a vector search silently dropping a hard constraint like budget. Fuzzy-intent questions ("I want to work in robotics") are embedded, matched against candidate programs via vector similarity, and then the same hard filters are re-applied via SQL before anything reaches the student. Vectors generate candidates; they never enforce a constraint.

3.2 Why tool-calling is the primary hallucination defense

The LLM never free-generates a fact. It calls `get_program_details(program_id)`, which returns typed JSON where every field carries its own `value`, `verification_status`, and `last_verified_date`. The system prompt requires the model to state only values present in that tool output, and to emit an explicit fallback — *"could not be verified from official university sources, please consult an Aspiro counselor"* — whenever a field is null or unverified. A cheap post-generation check (flag any number or date absent from the

tool payload) is a secondary net, not the primary defense.

4 Phased Roadmap

Table 2: Four-phase rollout. Phases 0–1 require no new infrastructure; Phases 2–3 need a team infrastructure decision (see §8).

Phase	Timeframe	Key Deliverables	Exit Trigger (move to next phase)
0 — Fix Now	Immediate, <1 day	Add `currency`, `last_verified_date`, `program_id` columns; freeze existing CSVs as immutable Bronze snapshots	Done as soon as the columns are backfilled — no dependency
1 — Automate	2–4 weeks	Cron-scheduled scraping; SHA-256 change detection; 3-tier normalization (lookup → RapidFuzz → LLM); Pandera validation; live FX via free API	Catalog passes ∼5 countries *or* manual re-verification becomes the bottleneck
2 — Structure	Follows Phase 1	Relational schema live in SQLite; 3-stage entity resolution; thin access API with freshness/trust filtering	Concurrent writers appear, *or* row count approaches 50k, *or* the LLM backend needs networked (non-embedded) access
3 — Ground the LLM	Parallel with / after Phase 2	Hybrid retrieval (SQL + pgvector); tool-calling grounding; deterministic FX tool calls; trust-tier disclosure in responses	Production launch readiness

5 Data Model

Phase 2 replaces the flat 17-column CSV with a normalized schema. Repeatable fields (prerequisites, tags, intakes) become real child rows instead of semicolon-flattened text, and every fact-bearing table carries its own source, trust tier, and verification status — freshness and governance are schema properties, not an afterthought.

Table	Purpose & Key Fields
`university`	One row per institution: name, country, city, website, `source_tier`.
`program`	Canonical program record: level, field, duration, `last_verified_date`, `verification_status`, `confidence_level`, `superseded_by`.
`program_alias`	Every scraped title variant ever seen for a program — the join point for entity resolution and the audit trail.
`fee_line`	One row per fee (tuition year 1, tuition total, application fee, deposit) in its **original currency**, never pre-converted.
`intake`	One row per intake term/deadline, with `term_normalized` mapped through the controlled vocabulary.
`requirement`	One row per prerequisite/must-have, with `is_confirmed_absent` distinguishing "checked, none exist" from "not yet checked."
`tag` / `program_tag`	Scholarships, campus features, accreditations — many-to-many, filtered clean of sitewide noise.
`source` / `term_dictionary`	Source registry with trust tier; growing lookup table that powers the normalization layer.

5.1 Program identity resolution

Three stages, cheapest first, so most rows never need the expensive path:

1. **Exact match** on normalized `(university, program name)` — auto-updates the existing record.
2. **Fuzzy match** via RapidFuzz — ≥ 0.95 similarity auto-merges; 0.85–0.95 is flagged for a short human review queue; below 0.85 is treated as a new program.
3. **Embedding similarity** — used only as a tiebreaker signal inside the flagged band, not as an auto-merge trigger. Deferred until catalog volume actually justifies the cost.

6 Source Governance

Every fact carries a numeric `source_tier`, stored *per record* (not just per university) so that a future Tier-1 fee line can outrank an existing Tier-5 line for the same program the moment it appears.

Tier	Source Type
1	Official university website
2	Official university API
3	Partner institution portal
4	Internal, human-verified Aspiro records
5	Third-party aggregator *(mastersportal.com — 100% of current data)*
6	User-generated content

Honest Flag

Every record collected to date sits at Tier 5. The governance machinery above only pays off once a second, higher-trust source enters the pipeline — until then, AspiroBrain has no conflicting values to arbitrate between, only a single (unverified-by-the-university) source to lean on. See Open Question 1 below.

7 Technology Stack

Layer	Now	Upgrade Trigger	Later
Orchestration	cron + Python scripts	>5 countries, or retries/alerting needed	Prefect OSS
Normalization	static lookup + RapidFuzz	unresolved-phrase volume grows	+ LLM fallback, written back to lookup table
Validation	Pandera	multi-engine pipelines, non-technical stakeholders	Great Expectations
Storage	SQLite + Litestream	concurrent writers, ∼50k+ rows, network access needed	Supabase (Postgres)
Entity resolution	exact → RapidFuzz	review queue too large for manual handling	+ embedding tiebreaker
Vector search	none	fuzzy-intent queries go live	pgvector, same Postgres instance
FX rates	Frankfurter API (primary), fawazahmed0 (fallback)	intraday-rate need	Paid tier (ExchangeRate-API, Open Exchange Rates)
LLM grounding	—	—	Tool-calling, field-level grounding (§3.2)

Table 3: Every layer has a stated upgrade trigger — nothing here is over-built for a 200-row catalog run by one intern.

8 Open Questions for the Team

Decisions Needed Before Phase 2–3

1. Is `mastersportal.com` meant to remain the sole data source, or is there a plan to reach Tier-1/Tier-2 sources? The trust-hierarchy design only earns its keep once more than one tier is actually represented in the data.
2. Who owns the fuzzy-match review queue (the 0.85–0.95 band) and flagged data changes — the intern, a counselor, or does this need a lightweight review UI?
3. What is the realistic timeline and budget for standing up Postgres/Supabase and an LLM backend? Phases 0–1 run entirely inside the current CSV/git workflow; Phases 2–3 need an infrastructure decision from the team.

9 Recommended Immediate Action

Next Step — No Infrastructure Required

Phase 0 can be completed today inside the existing `Data Collection/` workflow: add `currency`, `last_verified_date`, and `program_id` columns, and freeze the current Netherlands and Malta CSVs as immutable Bronze snapshots. This is roughly half a day of work and removes the schema gaps that get exponentially more expensive to backfill once the catalog grows past a handful of countries.

10 Further Reading

Sources surfaced during the research pass behind this plan:

- Frankfurter API
- fawazahmed0/exchange-api (GitHub)
- Decoding Data Orchestration Tools: Prefect, Dagster, Airflow, Mage
- Prefect vs Dagster
- RapidFuzz — A Fast, Flexible, Modern Library
- The Data Validation Landscape in 2025
- Great Expectations vs Pandera
- SQLite vs Postgres for the Solo Founder in 2026
- SQLite vs Supabase for Solo Developers
- Fuzzy Matching 101: The Complete Guide to Accurate Data Matching

Prepared as part of the AspiroWork data internship, grounded in the live mastersportal.com scraping pipeline (Netherlands + Malta, 200 programs, 10 July 2026).